

Yaw Damping Design

Table of Contents

1. Introduction
 - 1.1. Scope and Assumptions
 2. How To Read This Document
 3. Document Health Snapshot
 4. Terms and Definitions
 - 4.1. Diagram Color and Shape Legend
 5. Architecture Intent View
 - 5.1. What This View Answers
 - 5.2. Authored View Diagram
 - 5.3. System Boundary Diagram
 - 5.4. System Decomposition Diagram
 - 5.5. Functional Group Manhattan Matrix
 - 5.6. Functional Groups and Units (View Scoped)
 6. Traceability Appendix
 - 6.1. Requirement Details
 - 6.2. Verification Inventory
 7. Gemara GRC Model
 8. System Composition & Hardware
 - 8.1. Hardware Items
 9. Reference Index
 - 9.1. Authored References
 - 9.2. Catalog References
 - 9.3. Inferred Runtime References
 - 9.4. Inferred Code References
 - 9.5. Verification References
-

1. Introduction

Bounded readiness-only example for a non-autonomous yaw damping software item.

1.1. Scope and Assumptions

- Functional Groups and Functional Units are the stable authored design backbone.
- Runtime elements and code elements are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- Requirements and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

- 1. Concept layer: functional and view-specific architecture narratives for human understanding.
- 2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

- 1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
- 2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
- 3. Traceability View: requirement mapping, evidence coverage, and confidence.
- 4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (VIEW-YAW-CONTROL)	unspecified	No authored status explanation provided.	1	1	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.

4. Terms and Definitions

Term	Definition
actor (TERM-ACTOR)	A person or operational role that interacts with functional units.
architecture decision (TERM-DECISION)	Authored architecture decision record with status, context, decision text, and consequences.
AsciiDoc diagram (TERM-ASCIIDOC-DIAGRAM)	Generated diagram block, usually Mermaid, that visualizes architecture relationships in the AsciiDoc publication.
AsciiDoc document (TERM-ASCIIDOC-DOCUMENT)	Human-readable generated architecture publication document assembled from authored model, inferred evidence, diagrams, references, and decisions.
AsciiDoc section (TERM-ASCIIDOC-SECTION)	Structured generated document section or row model used to render authored and inferred architecture content.
attack vector (TERM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
authored mapping (TERM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.

Term	Definition
bounded rudder command (TERM-RUDDER-COMMAND)	A command constrained to the allocated authority.
catalog (TERM-CATALOG)	Controlled vocabulary document used to normalize model, requirement, and generated-document terminology.
catalog entry (TERM-CATALOG-ENTRY)	A stable catalog term with a definition and aliases for linting, linking, and generated references.
CLI command (TERM-CLI-COMMAND)	Command-line entrypoint that coordinates loading, validation, generation, and file output for an engineering-model workflow.
code element (TERM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
compliance mapping (TERM-COMPLIANCE-MAPPING)	Authored mapping that links a model control to selected compliance controls, model scope, implementation status, evidence, and responsible roles.
control (TERM-CONTROL)	An authored security, policy, or operational control used to constrain behavior and reduce risk.
control verification (TERM-CONTROL-VERIFICATION)	Authored control verification record with method, status, threat/risk scope, findings, and evidence.
data object (TERM-DATA-OBJECT)	An authored data artifact or contract shape traced across flow, interface, and implementation boundaries.
deployment target (TERM-DEPLOYMENT-TARGET)	An authored deployment destination such as environment, cluster, namespace, or registry scope.
design document (TERM-DESIGN)	Authored design narrative organized by model entities and architecture views.
design view (TERM-DESIGN-VIEW)	View-scoped design narrative attached to an authored functional group or functional unit.
event (TERM-EVENT)	An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes.
evidence (TERM-EVIDENCE)	A file, artifact, or observation used to support control, risk, threat, or verification claims.
flight control (FG-FLIGHT-CONTROL)	Functions that support controlled aircraft motion.
flow (TERM-FLOW)	An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps.

Term	Definition
flow step (TERM-FLOW-STEP)	A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions.
functional group (TERM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
functional unit (TERM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
generation workflow (TERM-GENERATION-WORKFLOW)	Orchestrated run that combines model loading, validation, projection, rendering, and artifact emission.
inference hint (TERM-INFERENCE-HINT)	Authored source and ownership configuration used to discover runtime, code, and verification evidence.
inferred mapping (TERM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
interface (TERM-INTERFACE)	An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit.
lint run (TERM-LINT-RUN)	Requirement lint configuration that controls parsing and quality checks for requirement text.
LOBSTER activity trace (TERM-LOBSTER-ACTIVITY-TRACE)	Generated LOBSTER activity JSON that links verification evidence back to formal requirement identifiers.
MCP server (TERM-MCP-SERVER)	Model Context Protocol server that exposes engineering-model operations to AI agents through JSON-RPC tools.
MCP stdio transport (TERM-MCP-STDIO-TRANSPORT)	Content-Length framed standard input/output transport used by the MCP command-line server.
MCP tool (TERM-MCP-TOOL)	Named MCP operation with a constrained input schema and structured response payload.
MCP tool response (TERM-MCP-TOOL-RESPONSE)	Structured MCP tool payload that includes success state, schema version, generated timestamp, and result data or validation error.
model (TERM-MODEL)	The authored architecture model root that composes functional structure, relationships, inference hints, and views.
model bundle (TERM-BUNDLE)	Loaded architecture, catalog, and companion documents resolved as one working model context.
Open OTM document (TERM-OPEN-OTM-DOCUMENT)	Open Threat Model JSON representation generated from the engineering model for interoperable threat-model exchange.

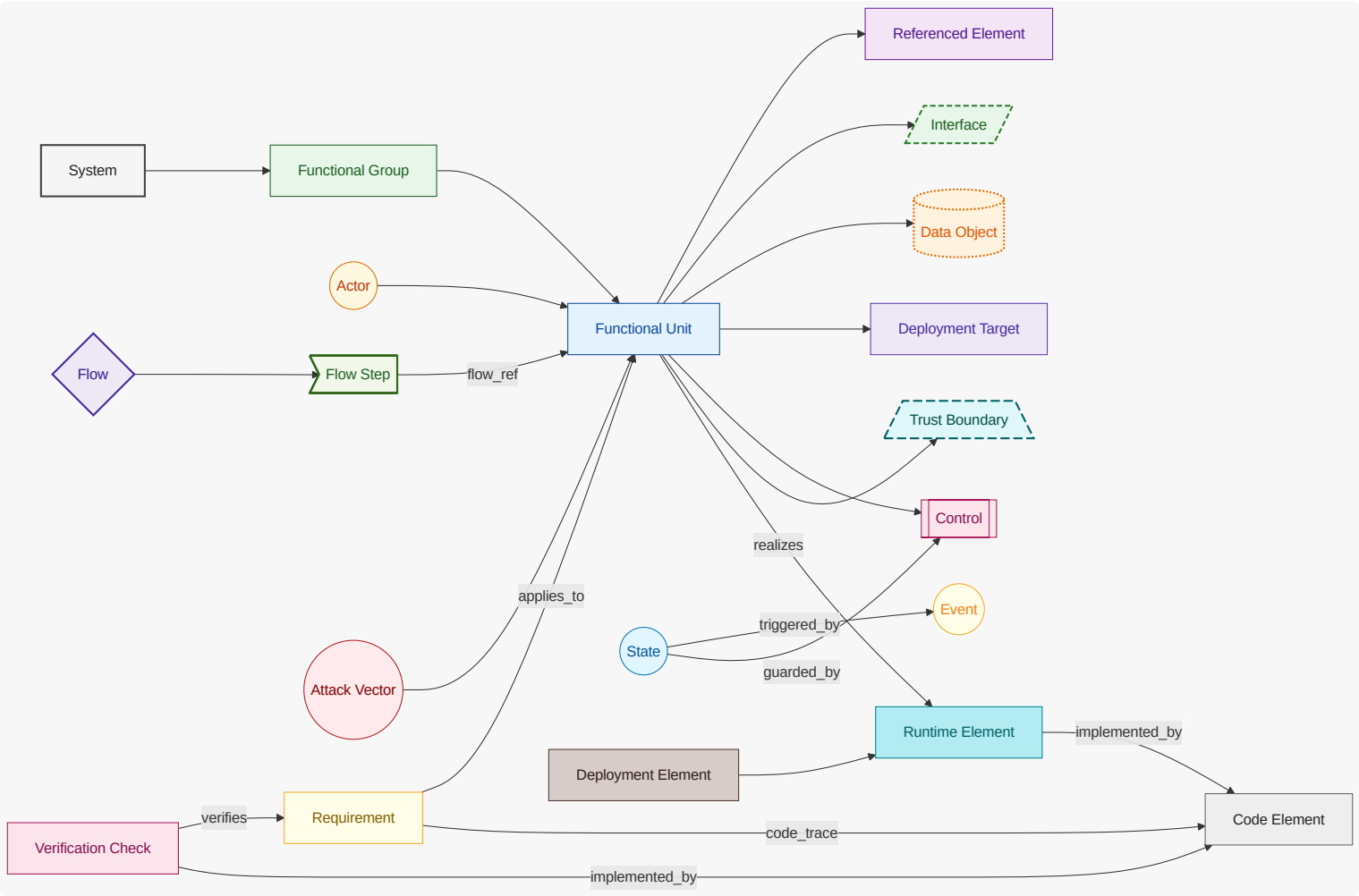
Term	Definition
operational (STATE-OPERATIONAL)	The software is able to calculate commands.
OSCAL assessment results (TERM-OSCAL-ASSESSMENT-RESULTS)	Generated OSCAL assessment results that summarize reviewed controls, verification findings, and modeled risks.
OSCAL POA&M (TERM-OSCAL-POAM)	Generated OSCAL plan of action and milestones that maps modeled POA&M items and related risks into compliance JSON.
OSCAL SSP (TERM-OSCAL-SSP)	Generated OSCAL system security plan that maps authored system metadata, components, controls, allocations, and evidence into SSP JSON.
POA&M item (TERM-POAM-ITEM)	Plan of action and milestones item linked to a modeled risk and supporting artifacts.
reference index (TERM-REFERENCE-INDEX)	Generated index of authored, catalog, runtime, code, and verification references with stable anchors and backlinks.
referenced element (TERM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
repository index (TERM-REPO-INDEX)	Bounded first-party source index used to answer model-aware file, requirement, control, and threat lookup queries.
requirement (TERM-REQUIREMENT)	A structured requirement statement that defines expected system behavior and maps to functional ownership.
risk (TERM-RISK)	Authored risk record with likelihood, impact, response, scope, controls, and evidence.
runtime element (TERM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
security context (TERM-SECURITY-CONTEXT)	Security-focused generated context view that groups owned functional units and shows external actors, references, flows, controls, and trust boundaries.
source block (TERM-SOURCE-BLOCK)	Stable source reference block that records file path, optional line span, kind, summary, and linked entity IDs.
state (TERM-STATE)	An authored lifecycle state used to model transition behavior, guards, and event-driven progression.
Structurizr DSL (TERM-STRUCTURIZR-DSL)	Generated Structurizr DSL workspace that projects authored architecture, relationships, dynamic views, and deployment metadata.
system safety assessment (REF-SYSTEM-SAFETY-ASSESSMENT)	External safety assessment that allocates the yaw damping objective.
systems engineer (ACT-SYSTEMS-ENGINEER)	Engineering stakeholder responsible for the system and software specifications.

Term	Definition
threat assumption (TERM-THREAT-ASSUMPTION)	Authored security assumption that records scope, rationale, owner, status, and evidence.
Threat Dragon document (TERM-THREAT-DRAGON-DOCUMENT)	Threat Dragon JSON representation generated from the engineering model for STRIDE threat-model review.
threat mitigation (TERM-THREAT-MITIGATION)	Authored mitigation record that links a threat scenario to a control, effectiveness, verification, and evidence.
threat model export (TERM-THREAT-MODEL-EXPORT)	Generated security artifact that translates authored architecture, flows, trust boundaries, threats, and mitigations into an external threat-model schema.
threat out-of-scope decision (TERM-THREAT-OUT-OF-SCOPE)	Authored decision that excludes a threat concern from current scope with reason, owner, expiry, and evidence.
threat scenario (TERM-THREAT-SCENARIO)	Authored threat narrative that connects attack vectors, scope, flows, controls, risks, and verification evidence.
trace marker (TERM-TRACE-MARKER)	Source-level marker such as TRLC-LINKS or ENGMODEL-LINKS used to connect declarations to requirements and model entities.
TRLC package (TERM-TRLC-PACKAGE)	Generated TRLC model and requirements package used for formal requirement traceability processing.
trust boundary (TERM-TRUST-BOUNDARY)	An authored trust separation zone that marks policy, identity, or security control boundaries.
upward linking (TERM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.
validation (TERM-VALIDATION)	Model quality gate that checks authored documents, references, relationship semantics, and requirement lint results.
validation diagnostic (TERM-VALIDATION-DIAGNOSTIC)	Structured validation finding with code, severity, message, and optional source path.
verification check (TERM-VERIFICATION-CHECK)	An inferred or authored verification artifact that validates requirement behavior with test evidence.
view (TERM-VIEW)	Authored projection configuration that selects roots, entity kinds, mappings, audience, and abstraction.
yaw damping software (FU-YAW-CONTROL)	Software function that filters yaw rate and limits its command.
yaw rate input (TERM-YAW-RATE)	Measured aircraft yaw angular rate.

Term	Definition
yaw rate input is received (EVT-YAW-RATE-RECEIVED)	A sampled yaw-rate value becomes available.

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



5. Architecture Intent View

Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

Authored status: unspecified

Authored scope and limitations: No authored status explanation provided.

5.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

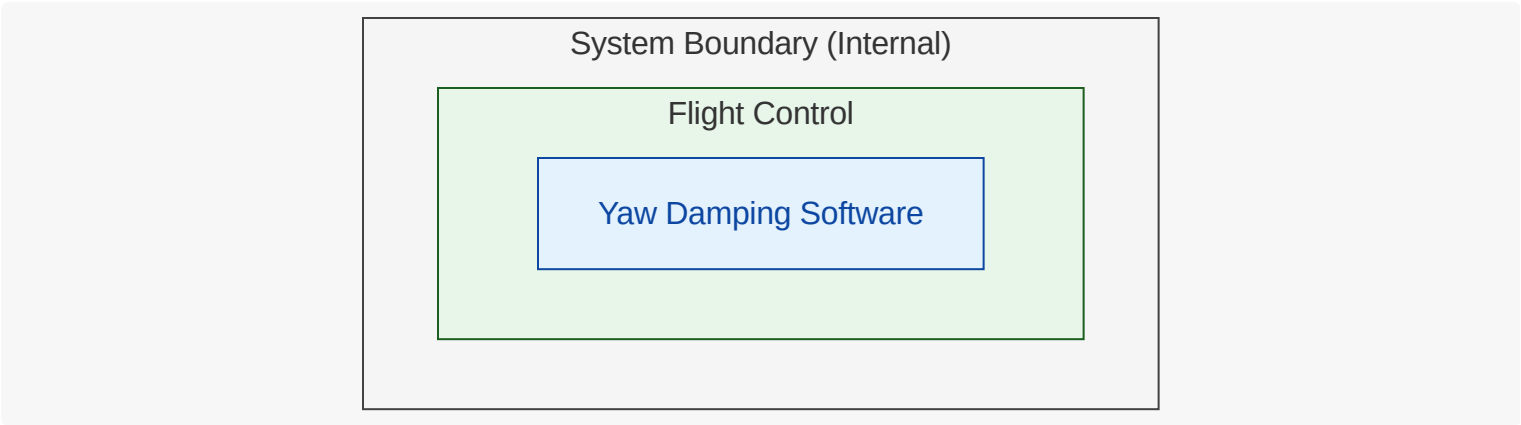
5.2. Authored View Diagram

This diagram follows the selected roots, depth, kinds and relationships of VIEW-YAW-CONTROL .



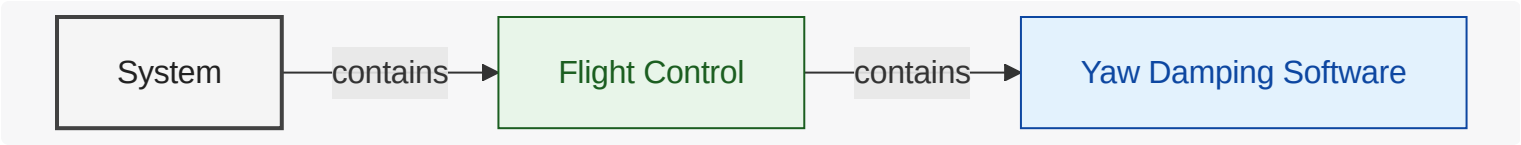
5.3. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the System Boundary (Internal) container; while actors and referenced dependencies are shown outside the boundary.



5.4. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.



5.5. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

Yaw Damping Software
Flight Control

5.6. Functional Groups and Units (View Scoped)

5.6.1. Flight Control (FG-FLIGHT-CONTROL)

Bounded flight-control functions.

Yaw Damping Software (FU-YAW-CONTROL)

Filters yaw rate and produces a bounded command.

Owned Responsibility

functional responsibility for yaw damping software

Key Collaborators

none

Linked Requirements

REQ-YD-HLR-001 , REQ-YD-LLR-001 , REQ-YD-SYS-001

6. Traceability Appendix

6.1. Requirement Details

6.1.1. REQ-YD-HLR-001

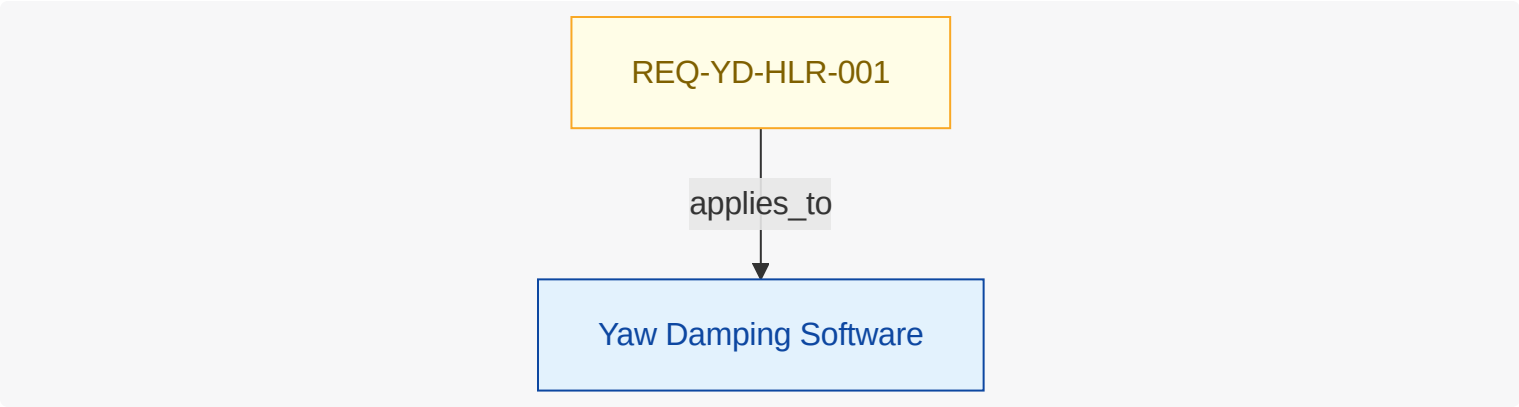
When yaw rate input is received , the yaw damping software shall filter the yaw rate input .

NOTE

High-level software requirement .

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



6.1.2. REQ-YD-LLR-001

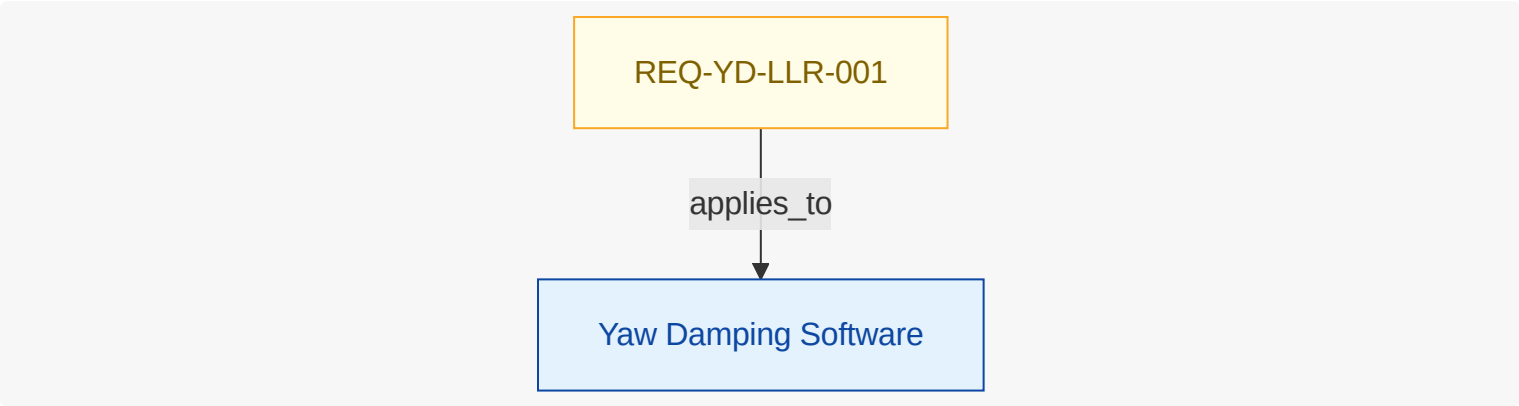
When yaw rate input is received , the yaw damping software shall limit the bounded rudder command .

NOTE

Derived low-level limit requiring safety feedback.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



6.1.3. REQ-YD-SYS-001

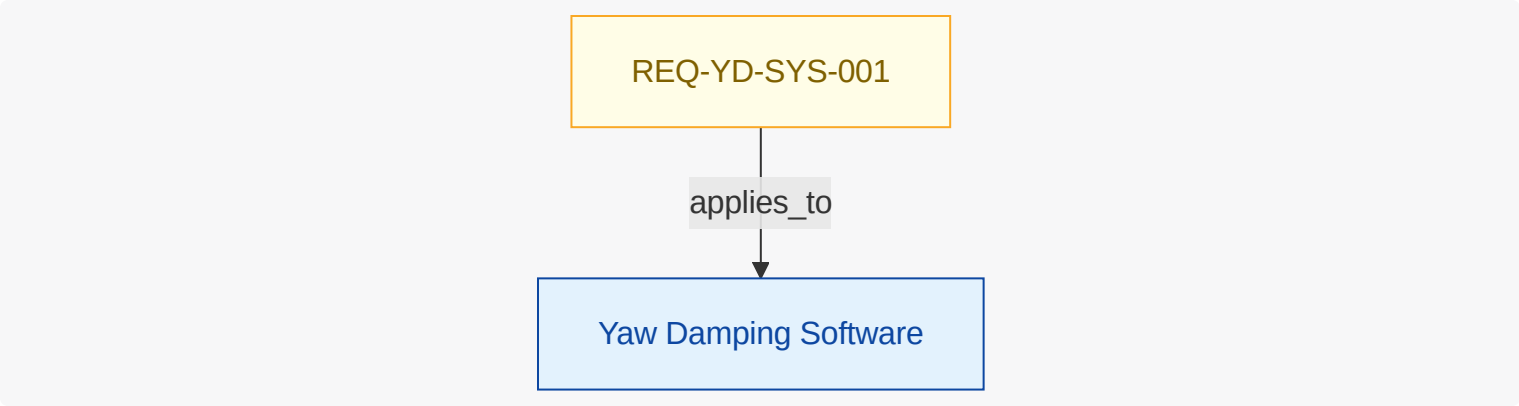
When yaw rate input is received , the yaw damping software shall compute a bounded rudder command .

NOTE

System-allocated behavior.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



6.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

7. Gemara GRC Model

This chapter renders the model as OpenSSF Gemara (<https://gemara.openssf.org>) documents. Each artifact is built with the official `go-gemara` SDK types and validated against the published Gemara CUE schemas. Generate the full YAML set with `enggemara` ; query it via the `gemara.*` MCP tools.

Gemara Layer	Artifact	Entries
L1	Vector Catalog	0
L1	Principle Catalog	5
L1	Guidance Catalog	0
L2	Capability Catalog	2
L2	Control Catalog	0
L2	Threat Catalog	0
L3	Risk Catalog	0
-	Lexicon	7

8. System Composition & Hardware

This chapter shows the system architecture across hardware and software: hardware items and their interfaces, the subsystems this system composes, and the allocation of this system’s requirements onto them. Subsystem references are downward-only and resolved from exact-version manifest dependencies, with optional workspace replacements.

8.1. Hardware Items

ID	Name	Kind	Hosts	Safety
HW-FLIGHT-CONTROL-COMPUTER	Flight Control Computer	computer		-

9. Reference Index

9.1. Authored References

IF-RUDDER-COMMAND

protocol=sampled-data; endpoint=output; schemaRef=n/a; owner=FU-YAW-CONTROL

Key	Value
Kind	Interface
Name	Rudder Command Output
Mentioned In	n/a

IF-YAW-RATE

protocol=sampled-data; endpoint=input; schemaRef=n/a; owner=FU-YAW-CONTROL

Key	Value
Kind	Interface
Name	Yaw Rate Input
Mentioned In	REQ-YD-HLR-001

REQ-YD-HLR-001

When yaw rate input is received, the yaw damping software shall filter the yaw rate input.

Key	Value
Kind	Requirement
Name	REQ-YD-HLR-001
Mentioned In	Architecture Intent View

REQ-YD-LLR-001

When yaw rate input is received, the yaw damping software shall limit the bounded rudder command.

Key	Value
Kind	Requirement
Name	REQ-YD-LLR-001

Key	Value
Mentioned In	Architecture Intent View

REQ-YD-SYS-001

When yaw rate input is received, the yaw damping software shall compute a bounded rudder command.

Key	Value
Kind	Requirement
Name	REQ-YD-SYS-001
Mentioned In	Architecture Intent View

9.2. Catalog References

ACT-SYSTEMS-ENGINEER

Engineering stakeholder responsible for the system and software specifications.

Key	Value
Kind	Catalog Actor
Name	systems engineer
Mentioned In	n/a

TERM-RUDDER-COMMAND

A command constrained to the allocated authority.

Key	Value
Kind	Catalog Data Term
Name	bounded rudder command
Mentioned In	REQ-YD-LLR-001 , REQ-YD-SYS-001

TERM-YAW-RATE

Measured aircraft yaw angular rate.

Key	Value
Kind	Catalog Data Term
Name	yaw rate input

Key	Value
Mentioned In	n/a

EVT-YAW-RATE-RECEIVED

A sampled yaw-rate value becomes available.

Key	Value
Kind	Catalog Event
Name	yaw rate input is received
Mentioned In	REQ-YD-HLR-001 , REQ-YD-LLR-001 , REQ-YD-SYS-001

FG-FLIGHT-CONTROL

Functions that support controlled aircraft motion.

Key	Value
Kind	Catalog Functional Group
Name	flight control
Mentioned In	n/a

FU-YAW-CONTROL

Software function that filters yaw rate and limits its command.

Key	Value
Kind	Catalog Functional Unit
Name	yaw damping software
Mentioned In	Architecture Intent View , REQ-YD-HLR-001 , REQ-YD-LLR-001 , REQ-YD-SYS-001

REF-SYSTEM-SAFETY-ASSESSMENT

External safety assessment that allocates the yaw damping objective.

Key	Value
Kind	Catalog Referenced Element
Name	system safety assessment
Mentioned In	n/a

STATE-OPERATIONAL

The software is able to calculate commands.

Key	Value
Kind	Catalog State
Name	operational
Mentioned In	n/a

TERM-ACTOR

A person or operational role that interacts with functional units.

Key	Value
Kind	Engineering Model Term
Name	actor
Mentioned In	n/a

TERM-ASCIIDOC-DIAGRAM

Generated diagram block, usually Mermaid, that visualizes architecture relationships in the AsciiDoc publication.

Key	Value
Kind	Engineering Model Term
Name	AsciiDoc diagram
Mentioned In	n/a

TERM-ASCIIDOC-DOCUMENT

Human-readable generated architecture publication document assembled from authored model, inferred evidence, diagrams, references, and decisions.

Key	Value
Kind	Engineering Model Term
Name	AsciiDoc document
Mentioned In	n/a

TERM-ASCIIDOC-SECTION

Structured generated document section or row model used to render authored and inferred architecture content.

Key	Value
Kind	Engineering Model Term
Name	AsciiDoc section
Mentioned In	n/a

TERM-ATTACK-VECTOR

A technical misuse or attack path that targets functional, referenced, or runtime elements.

Key	Value
Kind	Engineering Model Term
Name	attack vector
Mentioned In	n/a

TERM-AUTHORED-MAPPING

An explicit relationship declared in architecture inputs between authored or referenced elements.

Key	Value
Kind	Engineering Model Term
Name	authored mapping
Mentioned In	n/a

TERM-BUNDLE

Loaded architecture, catalog, and companion documents resolved as one working model context.

Key	Value
Kind	Engineering Model Term
Name	model bundle
Mentioned In	n/a

TERM-CATALOG

Controlled vocabulary document used to normalize model, requirement, and generated-document terminology.

Key	Value
Kind	Engineering Model Term

Key	Value
Name	catalog
Mentioned In	n/a

TERM-CATALOG-ENTRY

A stable catalog term with a definition and aliases for linting, linking, and generated references.

Key	Value
Kind	Engineering Model Term
Name	catalog entry
Mentioned In	n/a

TERM-CLI-COMMAND

Command-line entrypoint that coordinates loading, validation, generation, and file output for an engineering-model workflow.

Key	Value
Kind	Engineering Model Term
Name	CLI command
Mentioned In	n/a

TERM-CODE-ELEMENT

An inferred code structure or ownership element discovered from source trees and build metadata.

Key	Value
Kind	Engineering Model Term
Name	code element
Mentioned In	n/a

TERM-COMPLIANCE-MAPPING

Authored mapping that links a model control to selected compliance controls, model scope, implementation status, evidence, and responsible roles.

Key	Value
Kind	Engineering Model Term

Key	Value
Name	compliance mapping
Mentioned In	n/a

TERM-CONTROL

An authored security, policy, or operational control used to constrain behavior and reduce risk.

Key	Value
Kind	Engineering Model Term
Name	control
Mentioned In	n/a

TERM-CONTROL-VERIFICATION

Authored control verification record with method, status, threat/risk scope, findings, and evidence.

Key	Value
Kind	Engineering Model Term
Name	control verification
Mentioned In	n/a

TERM-DATA-OBJECT

An authored data artifact or contract shape traced across flow, interface, and implementation boundaries.

Key	Value
Kind	Engineering Model Term
Name	data object
Mentioned In	n/a

TERM-DECISION

Authored architecture decision record with status, context, decision text, and consequences.

Key	Value
Kind	Engineering Model Term
Name	architecture decision

Key	Value
Mentioned In	n/a

TERM-DEPLOYMENT-TARGET

An authored deployment destination such as environment, cluster, namespace, or registry scope.

Key	Value
Kind	Engineering Model Term
Name	deployment target
Mentioned In	n/a

TERM-DESIGN

Authored design narrative organized by model entities and architecture views.

Key	Value
Kind	Engineering Model Term
Name	design document
Mentioned In	n/a

TERM-DESIGN-VIEW

View-scoped design narrative attached to an authored functional group or functional unit.

Key	Value
Kind	Engineering Model Term
Name	design view
Mentioned In	n/a

TERM-EVENT

An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes.

Key	Value
Kind	Engineering Model Term
Name	event
Mentioned In	n/a

TERM-EVIDENCE

A file, artifact, or observation used to support control, risk, threat, or verification claims.

Key	Value
Kind	Engineering Model Term
Name	evidence
Mentioned In	n/a

TERM-FLOW

An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps.

Key	Value
Kind	Engineering Model Term
Name	flow
Mentioned In	n/a

TERM-FLOW-STEP

A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions.

Key	Value
Kind	Engineering Model Term
Name	flow step
Mentioned In	n/a

TERM-FUNCTIONAL-GROUP

A major authored capability area that groups related functional units.

Key	Value
Kind	Engineering Model Term
Name	functional group
Mentioned In	n/a

TERM-FUNCTIONAL-UNIT

An authored working unit inside a functional group that owns specific behavior.

Key	Value
Kind	Engineering Model Term
Name	functional unit
Mentioned In	n/a

TERM-GENERATION-WORKFLOW

Orchestrated run that combines model loading, validation, projection, rendering, and artifact emission.

Key	Value
Kind	Engineering Model Term
Name	generation workflow
Mentioned In	n/a

TERM-INFERENCE-HINT

Authored source and ownership configuration used to discover runtime, code, and verification evidence.

Key	Value
Kind	Engineering Model Term
Name	inference hint
Mentioned In	n/a

TERM-INFERRED-MAPPING

A discovered relationship that links inferred runtime/code elements upward to authored design.

Key	Value
Kind	Engineering Model Term
Name	inferred mapping
Mentioned In	n/a

TERM-INTERFACE

An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit.

Key	Value
Kind	Engineering Model Term

Key	Value
Name	interface
Mentioned In	n/a

TERM-LINT-RUN

Requirement lint configuration that controls parsing and quality checks for requirement text.

Key	Value
Kind	Engineering Model Term
Name	lint run
Mentioned In	n/a

TERM-LOBSTER-ACTIVITY-TRACE

Generated LOBSTER activity JSON that links verification evidence back to formal requirement identifiers.

Key	Value
Kind	Engineering Model Term
Name	LOBSTER activity trace
Mentioned In	n/a

TERM-MCP-SERVER

Model Context Protocol server that exposes engineering-model operations to AI agents through JSON-RPC tools.

Key	Value
Kind	Engineering Model Term
Name	MCP server
Mentioned In	n/a

TERM-MCP-STDIO-TRANSPORT

Content-Length framed standard input/output transport used by the MCP command-line server.

Key	Value
Kind	Engineering Model Term
Name	MCP stdio transport

Key	Value
Mentioned In	n/a

TERM-MCP-TOOL

Named MCP operation with a constrained input schema and structured response payload.

Key	Value
Kind	Engineering Model Term
Name	MCP tool
Mentioned In	n/a

TERM-MCP-TOOL-RESPONSE

Structured MCP tool payload that includes success state, schema version, generated timestamp, and result data or validation error.

Key	Value
Kind	Engineering Model Term
Name	MCP tool response
Mentioned In	n/a

TERM-MODEL

The authored architecture model root that composes functional structure, relationships, inference hints, and views.

Key	Value
Kind	Engineering Model Term
Name	model
Mentioned In	n/a

TERM-OPEN-OTM-DOCUMENT

Open Threat Model JSON representation generated from the engineering model for interoperable threat-model exchange.

Key	Value
Kind	Engineering Model Term
Name	Open OTM document

Key	Value
Mentioned In	n/a

TERM-OSCAL-ASSESSMENT-RESULTS

Generated OSCAL assessment results that summarize reviewed controls, verification findings, and modeled risks.

Key	Value
Kind	Engineering Model Term
Name	OSCAL assessment results
Mentioned In	n/a

TERM-OSCAL-POAM

Generated OSCAL plan of action and milestones that maps modeled POA&M items and related risks into compliance JSON.

Key	Value
Kind	Engineering Model Term
Name	OSCAL POA&M
Mentioned In	n/a

TERM-OSCAL-SSP

Generated OSCAL system security plan that maps authored system metadata, components, controls, allocations, and evidence into SSP JSON.

Key	Value
Kind	Engineering Model Term
Name	OSCAL SSP
Mentioned In	n/a

TERM-POAM-ITEM

Plan of action and milestones item linked to a modeled risk and supporting artifacts.

Key	Value
Kind	Engineering Model Term
Name	POA&M item

Key	Value
Mentioned In	n/a

TERM-REFERENCE-INDEX

Generated index of authored, catalog, runtime, code, and verification references with stable anchors and backlinks.

Key	Value
Kind	Engineering Model Term
Name	reference index
Mentioned In	n/a

TERM-REFERENCED-ELEMENT

An architecture-relevant external, platform, or third-party dependency represented by role.

Key	Value
Kind	Engineering Model Term
Name	referenced element
Mentioned In	n/a

TERM-REPO-INDEX

Bounded first-party source index used to answer model-aware file, requirement, control, and threat lookup queries.

Key	Value
Kind	Engineering Model Term
Name	repository index
Mentioned In	n/a

TERM-REQUIREMENT

A structured requirement statement that defines expected system behavior and maps to functional ownership.

Key	Value
Kind	Engineering Model Term
Name	requirement
Mentioned In	REQ-YD-HLR-001

TERM-RISK

Authored risk record with likelihood, impact, response, scope, controls, and evidence.

Key	Value
Kind	Engineering Model Term
Name	risk
Mentioned In	n/a

TERM-RUNTIME-ELEMENT

An inferred runtime realization element discovered from infrastructure and deployment sources.

Key	Value
Kind	Engineering Model Term
Name	runtime element
Mentioned In	n/a

TERM-SECURITY-CONTEXT

Security-focused generated context view that groups owned functional units and shows external actors, references, flows, controls, and trust boundaries.

Key	Value
Kind	Engineering Model Term
Name	security context
Mentioned In	n/a

TERM-SOURCE-BLOCK

Stable source reference block that records file path, optional line span, kind, summary, and linked entity IDs.

Key	Value
Kind	Engineering Model Term
Name	source block
Mentioned In	n/a

TERM-STATE

An authored lifecycle state used to model transition behavior, guards, and event-driven progression.

Key	Value
Kind	Engineering Model Term
Name	state
Mentioned In	n/a

TERM-STRUCTURIZR-DSL

Generated Structurizr DSL workspace that projects authored architecture, relationships, dynamic views, and deployment metadata.

Key	Value
Kind	Engineering Model Term
Name	Structurizr DSL
Mentioned In	n/a

TERM-THREAT-ASSUMPTION

Authored security assumption that records scope, rationale, owner, status, and evidence.

Key	Value
Kind	Engineering Model Term
Name	threat assumption
Mentioned In	n/a

TERM-THREAT-DRAGON-DOCUMENT

Threat Dragon JSON representation generated from the engineering model for STRIDE threat-model review.

Key	Value
Kind	Engineering Model Term
Name	Threat Dragon document
Mentioned In	n/a

TERM-THREAT-MITIGATION

Authored mitigation record that links a threat scenario to a control, effectiveness, verification, and evidence.

Key	Value
Kind	Engineering Model Term
Name	threat mitigation
Mentioned In	n/a

TERM-THREAT-MODEL-EXPORT

Generated security artifact that translates authored architecture, flows, trust boundaries, threats, and mitigations into an external threat-model schema.

Key	Value
Kind	Engineering Model Term
Name	threat model export
Mentioned In	n/a

TERM-THREAT-OUT-OF-SCOPE

Authored decision that excludes a threat concern from current scope with reason, owner, expiry, and evidence.

Key	Value
Kind	Engineering Model Term
Name	threat out-of-scope decision
Mentioned In	n/a

TERM-THREAT-SCENARIO

Authored threat narrative that connects attack vectors, scope, flows, controls, risks, and verification evidence.

Key	Value
Kind	Engineering Model Term
Name	threat scenario
Mentioned In	n/a

TERM-TRACE-MARKER

Source-level marker such as TRLC-LINKS or ENGMODEL-LINKS used to connect declarations to requirements and model entities.

Key	Value
Kind	Engineering Model Term
Name	trace marker
Mentioned In	n/a

TERM-TRLC-PACKAGE

Generated TRLC model and requirements package used for formal requirement traceability processing.

Key	Value
Kind	Engineering Model Term
Name	TRLC package
Mentioned In	n/a

TERM-TRUST-BOUNDARY

An authored trust separation zone that marks policy, identity, or security control boundaries.

Key	Value
Kind	Engineering Model Term
Name	trust boundary
Mentioned In	n/a

TERM-UPWARD-LINKING

Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

Key	Value
Kind	Engineering Model Term
Name	upward linking
Mentioned In	n/a

TERM-VALIDATION

Model quality gate that checks authored documents, references, relationship semantics, and requirement lint results.

Key	Value
Kind	Engineering Model Term
Name	validation
Mentioned In	n/a

TERM-VALIDATION-DIAGNOSTIC

Structured validation finding with code, severity, message, and optional source path.

Key	Value
Kind	Engineering Model Term
Name	validation diagnostic
Mentioned In	n/a

TERM-VERIFICATION-CHECK

An inferred or authored verification artifact that validates requirement behavior with test evidence.

Key	Value
Kind	Engineering Model Term
Name	verification check
Mentioned In	n/a

TERM-VIEW

Authored projection configuration that selects roots, entity kinds, mappings, audience, and abstraction.

Key	Value
Kind	Engineering Model Term
Name	view
Mentioned In	n/a

9.3. Inferred Runtime References

9.4. Inferred Code References

9.5. Verification References